

EXPERIMENT 3

Aim

Socket Programming for Transport Layer Packets (TCP Client)

Theory

In a client-server architecture, the **client** acts as the active process that initiates communication by sending a request to a passive server running on a known IP address and port number.

At the Transport Layer, the TCP Client provides **Active Open** socket functionality. It guarantees data delivery by using sequence numbers, acknowledgments, and checksums over a virtual connection stream.

Client-Side Execution Steps:

1. **Socket Creation (socket()):** The client initializes an endpoint socket specifying the address family (AF_INET for IPv4) and protocol type (SOCK_STREAM for TCP).
2. **Server Addressing:** The client configures a socket address structure (sockaddr_in) specifying the target server's destination **IP Address** and **Port Number**.
3. **Initiating Connection (connect()):** The client calls connect(), which triggers the **TCP Three-Way Handshake** with the listening server:
 - **SYN:** Client sends a synchronization packet to the server.
 - **SYN-ACK:** Server responds with synchronization and acknowledgment.
 - **ACK:** Client sends an acknowledgment back to confirm the connection.
4. **Data Transmission (send() / recv() or Streams):** Once established, the client sends requests or reads responses through full-duplex socket streams.
5. **Termination (close()):** Upon completing the interaction, the client issues a close() call to gracefully close the socket connection and release kernel resources.

```
lab-03-17@lab-03-17-OptiPlex-3280-A10:~$ vim client_1290.c
lab-03-17@lab-03-17-OptiPlex-3280-A10:~$ gcc client_1290.c -o client
lab-03-17@lab-03-17-OptiPlex-3280-A10:~$ ./client
Enter message: Hello, This is the request sent by the client
Echo from server: Hello, This is the request sent by the client
Enter message: Hello Server
Echo from server: Hello Server
Enter message: 
```

